PocketDoctor Mark 4: Master Engineering & Firmware Manual

Vaidik Khurana

Systems Architect & Developer | Project Gateway: https://pocdoc.vaidik.co
Target Platform: ESP32 Microcontroller + Groq LLaMA 3.3 70B AI | License: MIT

TABLE OF CONTENTS

1. EXECUTIVE SUMMARY & PROBLEM STATEMENT

1.1 The Problem Statement

In modern healthcare infrastructure—particularly in developing regions, rural communities, and over-burdened medical triage centers—access to immediate clinical diagnostic guidance is severely constrained. Patients frequently face long waiting times for basic medical triage, leading to delayed treatment for acute conditions or unnecessary overcrowding in emergency rooms for non-critical complaints.

As detailed on the project gateway https://pocdoc.vaidik.co, existing digital health tools often rely heavily on constant high-bandwidth smartphone app connectivity, bulky computer hardware, or expensive proprietary software. These requirements make existing solutions impractical for immediate point-of-care deployment in resource-constrained environments.

1.2 The PocketDoctor Solution

PocketDoctor Mark 4 addresses this challenge by establishing an autonomous, low-cost, handheld point-of-care medical triage assistant. Powered by an ESP32 microcontroller and Groq's high-speed llama-3.3-70b-versatile cloud model, PocketDoctor provides:

• **Autonomous Physical Interface**: Direct physical interaction using a 128x64 SSD1306 OLED display and 2-axis joystick, enabling patients to input symptoms directly without requiring a smartphone.

• **Persistent Patient Profiling**: Stores up to 3 user profiles in 512-byte EEPROM memory (Age, Height, Weight, Intoxication history, Medical history, and Treatment preference).

• **Dynamic 10-Turn AI Dialogue**: Conducts an adaptive Q&A; dialogue tailored to the patient's demographics and symptoms via Groq API.

• **Local Web Reporting**: Synthesizes structured clinical assessments and hosts a responsive HTML report directly on the ESP32 over local Wi-Fi for browser viewing and printing.

2. SYSTEM ARCHITECTURE & HARDWARE TOPOLOGY

The ESP32 application operates on a single-core main loop architecture combining synchronous UI rendering, non-blocking input handling, asynchronous web client processing, and HTTP REST API communication with Groq Cloud.

3. COMPREHENSIVE CODEBASE & FIRMWARE BREAKDOWN (`src/main.ino`)

This section provides an engineering walkthrough of the firmware logic implemented in src/main.ino.

3.1 Global Configuration & Driver Allocations (Lines 1–150)

The code imports essential driver header files: WiFi.h and HTTPClient.h for networking; ArduinoJson.h for JSON parsing; Wire.h, Adafruit_GFX.h, and Adafruit_SSD1306.h for display drivers; EEPROM.h for flash memory access; and WebServer.h for Port 80 HTTP server functionality. OLED parameters are instantiated at 128x64 pixels on I2C address 0x3C. Joystick ADC pins are assigned to VRx (GPIO 34) and VRy (GPIO 35) with threshold joyThreshold = 1000 and debounceDelay = 200 ms. Pushbutton SW is assigned to GPIO 32.

3.2 EEPROM Storage & Profile Serialization Protocols (Lines 170–267)

Memory persistence is managed via four dedicated functions:
• loadWiFiConfig(): Initializes 512-byte EEPROM space and evaluates byte address 0. If magic == 0xABCD, loads stored SSID and password; otherwise flags uninitialized and falls back to default network credentials (NKJIO2.4).
• saveWiFiConfig(ssid, pass): Sets magic = 0xABCD, writes null-terminated network strings into the Config struct, and executes EEPROM.commit().
• saveProfilesToEEPROM(): Commits profile records starting at address 100 (PROFILES_START). Writes profileCount, then serializes string lengths, ASCII byte sequences, numerical metrics (age, height, weight), and boolean flags.
• loadProfilesFromEEPROM(): Reads byte 100 and deserializes stored byte streams back into the savedProfiles[3] struct array.

3.3 On-Device Physical UI Engine & Form Builders (Lines 270–632)

Physical UI navigation is governed by handleProfileMenu() and handleProfileCreation():
• **Profile Selection Menu**: Renders saved profiles or 'Create New'. Joystick Y-axis movement increments/decrements cursor index; button press selects active profile or initiates profile creation.
• **Interactive Form Builder**: Executes across 8 dynamic field states:
- *Field 0 (Name Entry)*: Letter selector using alphabet[] = "ABCDEFGHIJKLMNOPQRSTUVWXYZ ". Joystick Y-axis scrolls letters; short button click appends letter; holding button for >2000 ms commits the name.
- *Fields 1–3 (Age, Height, Weight)*: Bounded numeric adjusters (Age: 1–120, Height: 50–250 cm, Weight: 20–300 kg).
- *Fields 4–6 (Intoxication, History, Treatment)*: Option pickers for intoxication status, medical history flag, and treatment modality (Allopathy, Homeopathy, Ayurvedic, All types).
- *Field 7 (Save Profile)*: Commits profile to array and updates EEPROM flash memory.

3.4 Groq Cloud AI Integration & JSON Payload Engine (Lines 1042–1468)

The AI engine interfaces with Groq Cloud via sendToGroq(userMessage, isFinalDiagnosis):
• Constructs dynamic JSON payloads formatted as {"model":"llama-3.3-70b-versatile","messages":[...],"temperature":0.7}.
• Sets HTTP headers (Content-Type: application/json, Authorization: Bearer) and executes HTTP POST request.
• Parses response choices using DynamicJsonDocument doc(4096). Incorporates connection drop retry logic (handling error codes -11 and -1).
• startGroqDiagnosisWithData(): Serializes patient context and reported symptoms into the initial prompt context.
• handleGeminiQuestions(): Executes the 10-turn Q&A; loop. Displays questions on OLED, reads Joystick YES/NO input, buffers responses into conversationHistory, and triggers final output extraction.
• parseDiagnosis(diagnosis): Extracts disease name, accuracy percentage, medicines, precautions, and criticality flag using substring boundary searching.

3.5 Embedded Web Server & REST Portal Handlers (Lines 634–984)

All web templates (DASHBOARD_HTML, WIFI_HTML, PROFILES_HTML, PROFILE_EDIT_HTML) are stored in flash memory using PROGMEM blocks to conserve SRAM. The server processes templates dynamically via processTemplate(), replacing %IP%, %UPTIME%, and %PROFILES% placeholders before serving HTTP 200 responses. Endpoints support Wi-Fi network updates (/wifi/update), Web Profile CRUD operations (/profiles, /profile/update, /profile/delete), and clinical HTML report viewing (/report).

3.6 System Core Setup & Main Control Loop (Lines 1513–1723)

• setup(): Initializes Serial console @ 115200 baud, verifies SSD1306 OLED display, configures joystick pin modes (INPUT_PULLUP), loads EEPROM configurations, connects to Wi-Fi network, starts web server, and displays animated splash screen graphics.
• loop(): Non-blocking execution loop. Continuously handles background web clients (server.handleClient()), then evaluates UI state flags (inProfileMenu, inProfileCreation, inSymptomSelection, inDurationSelection, inGeminiQuestions).

4. HARDWARE SPECIFICATIONS & PIN ASSIGNMENTS

Peripheral	Pin Signal	ESP32 GPIO	Operational Specifications
SSD1306 OLED	SDA	GPIO 21	3.3V Logic | I2C Serial Data
SSD1306 OLED	SCL	GPIO 22	I2C Serial Clock (Address 0x3C)
KY-023 Joystick	VRx	GPIO 34	Analog X-Axis (12-bit ADC1_CH6)
KY-023 Joystick	VRy	GPIO 35	Analog Y-Axis (12-bit ADC1_CH7)
KY-023 Joystick	SW	GPIO 32	Digital Push Button (INPUT_PULLUP)
EEPROM Flash	Internal	NVS Flash	512 Bytes Reserved Non-volatile Storage

***Table I**: Hardware Peripheral Assignments and Operational Specifications*

5. GROQ AI DIALOGUE PROTOCOL & OUTPUT EXTRACTION

PocketDoctor executes a multi-turn JSON dialogue with Groq's llama-3.3-70b-versatile model. After 10 turns, the AI output is parsed into structured clinical variables (Disease, Accuracy, Medicines, Precautions, Critical Flag), rendering a short summary on screen and building a responsive HTML medical report.

6. EMBEDDED WEB SERVER & REMOTE MANAGEMENT PORTAL

Endpoint Route	Method	Handler Function	Operational Description
/ & /portal	GET	handleDashboard()	Real-time system status, local IP, and uptime dashboard.
/wifi	GET	handleWifi()	Wi-Fi network configuration update page.
/wifi/update	POST	handleWifiUpdate()	Saves new credentials to EEPROM & reboots device.
/profiles	GET	handleProfiles()	Patient profiles web management list dashboard.
/profile/edit	GET	handleProfileEdit()	Web form for creating or editing patient profile records.
/profile/update	POST	handleProfileUpdate()	Validates submitted fields and writes to EEPROM.
/profile/delete	GET	handleProfileDelete()	Deletes patient profile record at given index.
/report	GET	handleReport()	Hosts responsive HTML clinical report.

***Table II**: Embedded Web Server RESTful Route Matrix*

7. EXPERIMENTAL METRICS & BENCHMARKS

- **Cold Boot Time**: 1.25 seconds.
- **Wi-Fi Connection Time**: 2.10 seconds.
- **Groq API Latency per Turn**: 480 – 720 ms.
- **HTML Report Generation Time**: 45 ms.
- **Peak SRAM Memory Consumption**: 112.4 KB / 320 KB.

8. SETUP, COMPILATION & DEPLOYMENT GUIDE

1. Open src/main.ino in Arduino IDE or VS Code with Arduino extension.
2. Configure Board to ESP32 Dev Module and Partition Scheme to Default 4MB with spiffs.
3. Compile and flash firmware to ESP32 board.
4. Monitor Serial console @ 115200 baud to view system initialization logs and local IP assignment.

9. LEGAL LICENSE & ATTESTATION